

Comparative Analysis of Surrogate Methods

Jesper Michiel Janssen*, Jesus Guillermo Falcon Cardona†

*Tecnologico de Monterrey, School of Engineering and Sciences

Email: a01803288@tec.mx

†Tecnologico de Monterrey, School of Engineering and Sciences

Email: jfalcon@tec.mx

Abstract—Neural Networks are at the center point of artificial intelligence and the most important tool for computer science in the future. Where these networks are generally manually configured, a new technique has surfaced, called Neural Architecture Search (NAS). NAS automates the generation of Neural Networks. This method is highly interesting. However, it contains one major flaw. The computational cost of training all networks generated. This comes with a large carbon footprint, only increasing as NAS could become a large part of our day-to-day life. Therefore, research on performance predictors have recently picked up. However, a robust comparative analysis of all state-of-the-art surrogate methods is absent to this day. Thus, this drove us to perform said analysis, containing four predictors. Namely, an XGBoost regressor (XGB), Support Vector Regressor (SVR), Radial Basis Function (RBF) Network, and a Multi-Layer-Perceptron (MLP). After generating optimal hyperparameters via a gridsearch, we found that generally the RBF outperformed the other predictors combined with a relatively fast runtime.

I. INTRODUCTION

Image restoration (IR) is a critical problem in numerous applications, spanning fields such as medical imaging [18], video surveillance [19], astronomy [10], and forensics [9]. The main objective of IR is to enhance image quality by removing corruption, noise, or blur, making it essential to define and solve challenging optimization problems. As IR can be characterized as a multi-objective optimization problem (MOP), there is a need to balance multiple, often conflicting objectives to improve the performance of IR systems. Achieving effective IR under realistic constraints—such as limited resources, op-

erational requirements, and quality measures—can lead to more robust applications.

With the rise of Artificial Intelligence (AI), IR has increasingly been tackled through Deep Neural Networks (DNNs)[11], [16], [5]. DNNs have demonstrated success across various fields, including healthcare, finance, and autonomous systems[15], [6], [7]. However, manually configuring DNN parameters and architecture is time-consuming and error-prone, often requiring expert knowledge. A promising solution is to automate the process through Neural Architecture Search (NAS) [20], which optimizes both the architecture and parameters of DNNs, aiming to find architectures that perform well on specific tasks.

In recent years, the energy demands and environmental impact of NAS have become increasingly important considerations. Studies like [17] highlight the significant carbon emissions associated with a single NAS run, which is comparable to the lifetime emissions of multiple cars. Most of these energy demands and environmental impact are grounded in the training and validation of these networks. This awareness has spurred interest in developing carbon-efficient NAS methods, particularly through the usage of a surrogate method to predict the quality of the networks without having to train and validate them. While there has been some research on surrogate-assisted NAS, especially in image classification [2], [8], [1], less attention has been given to IR applications and sound comparative analysis of all possible predictors is lacking.

To address these issues, this study focuses on

developing the four most common state-of-the-art surrogate methods and concurrently performing a comparative analysis. To do this partially trained networks will be used for training the models with Peak Signal To Noise Ratio (PSNR) as ground truth. The goal is to create a predictor method that alleviates the computational burden of NAS, enabling IR applications that align with global efforts to minimize the environmental impact of AI.

II. RELATED WORKS

Evaluation of quality of the architectures is an essential part of NAS. Subsequently, it requires the most energy consumption and thus is believed to be the largest cause of carbon emissions within a NAS framework [12], [13], [21], [4], [?] ADD ELSKEN!!!). In recent history various methods have been developed and studied to circumvent this issue. Inheritance techniques like weights sharing is an interesting option to accelerate the quality estimates Zhu et al., 2019; Chen et al., 2020a; Wei et al., 2016). Another rational solution deem be, low fidelity training to estimate performance more efficiently [21], [4]. Another possibility is to partition the data set [3]. However, these techniques could introduce bias and cause inaccurate estimates. Lastly, quality predictors (i.e. regression tree models, Deep Neural Networks and gaussian processes) are a propose an interesting surrogate to handle the issue of energy expensive training. These methods predict the performance of networks using the variables of said networks as input. These computationally cheap predictors could lead to significant steps in NAS in day-to-day life.

In recent years multiple surrogate-assisted NAS approaches have been proposed. [14] introduced a comparative analysis of many regressors, where they found Support Vector Regression to be the most promising predictor, with XGBoost as a notable second. However, the use of their train-test (only 3.6% data for testing) could introduce inaccuracies. Also, two other prominent surrogate methods, namely Radial Basis Function Network (RBFs) and Multi-Layer-Perceptron (MLP) are left

out of the comparative analysis. Therefore, we aim to investigate and compare the above-mentioned predictors. Our goal is to not only validate [14] results on proper a proper training and test split, but also to add RBF and MLP predictors to the equation.

III. METHODOLOGY

In this section the methodology of this study will be described, concerning all intrinsics and mathematical formulation of the four predictors, the data set used, the gridsearch performed on the predictors and, lastly, the preprocessing of the data.

A. XGBoost

1) *Overview:* XGBoost (**X**treme **G**radient **B**oosting) is an ensemble learning method that builds a series of decision trees sequentially, where each subsequent tree attempts to correct the errors of its predecessors. The model optimizes a regularized objective function to prevent overfitting.

2) *Mathematical Formulation:* The objective function of XGBoost is composed of two parts: the loss function and the regularization term.

$$\text{Obj}(\Theta) = \sum_{i=1}^n l(y_i, \hat{y}_i) + \sum_{k=1}^K \Omega(f_k) \quad (1)$$

Where:

- n is the number of training samples.
- l is the loss function measuring the difference between the true value y_i and the prediction $\hat{y}_i$.
- K is the number of trees.
- $\Omega(f_k)$ is the regularization term for the k -th tree to control model complexity.

The regularization term is defined as:

$$\Omega(f) = \gamma T + \frac{1}{2} \lambda \sum_{j=1}^T w_j^2 \quad (2)$$

Where:

- T is the number of leaves in the tree.
- w_j is the weight of the j -th leaf.
- γ and λ are regularization parameters controlling the complexity of the tree.

3) *Gradient Boosting Framework*: XGBoost uses gradient boosting where trees are added to the ensemble to minimize the residual errors of the existing trees. The prediction is given by:

$$\hat{y}_i = \sum_{k=1}^K f_k(x_i) \quad (3)$$

Each tree f_k is fitted on the gradient of the loss function with respect to the current predictions.

B. Support Vector Regression (SVR)

1) *Overview*: Support Vector Regression extends the principles of Support Vector Machines (SVM) to regression problems. SVR aims to find a function that deviates from the actual observed targets by a value no greater than a specified threshold.

2) *Mathematical Formulation*: The SVR optimization problem can be formulated as:

$$\min_{\mathbf{w}, b, \xi, \xi^*} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n (\xi_i + \xi_i^*) \quad (4)$$

$$\text{subject to } y_i - (\mathbf{w}^\top \phi(x_i) + b) \leq \epsilon + \xi_i \quad (5)$$

$$(\mathbf{w}^\top \phi(x_i) + b) - y_i \leq \epsilon + \xi_i^* \quad (6)$$

$$\xi_i, \xi_i^* \geq 0 \quad \forall i \quad (7)$$

Where:

- $\mathbf{w}$ and b define the regression hyperplane.
- $\phi(x_i)$ is the feature mapping to a higher-dimensional space.
- ϵ defines the width of the ϵ -tube within which no penalty is associated in the training loss function with points predicted within a distance ϵ from the actual value.
- C is the regularization parameter controlling the trade-off between the model complexity and the amount up to which deviations larger than ϵ are tolerated.
- ξ_i, ξ_i^* are slack variables allowing for deviations outside the ϵ -tube.

3) *Kernel Trick*: To handle non-linear relationships, SVR utilizes kernel functions. For example, the Radial Basis Function (RBF) kernel is defined as:

$$K(x_i, x_j) = \exp(-\gamma \|x_i - x_j\|^2) \quad (8)$$

Where $\gamma > 0$ is a kernel parameter.

C. Radial Basis Function (RBF) Network

1) *Overview*: An RBF Network is a type of artificial neural network that uses radial basis functions as activation functions. It is typically composed of three layers: input, hidden (RBF), and output.

2) *Mathematical Formulation*: The output of an RBF Network for an input vector $\mathbf{x}$ is given by:

$$\hat{y}(\mathbf{x}) = \sum_{j=1}^M w_j \phi(\|\mathbf{x} - \mathbf{c}_j\|) \quad (9)$$

Where:

- M is the number of RBF neurons in the hidden layer.
- w_j is the weight connecting the j -th RBF neuron to the output neuron.
- $\mathbf{c}_j$ is the center of the j -th RBF neuron.
- $\phi(r)$ is the radial basis function, typically a Gaussian:

$$\phi(r) = \exp(-\beta R^2) \quad (10)$$

Where $\beta > 0$ determines the width of the Gaussian.

3) *Training Process*: Training involves determining the centers $\mathbf{c}_j$, parameters β , and weights w_j . This can be achieved using methods such as k-means clustering for centers and linear regression for weights.

D. Multi-Layer Perceptron (MLP)

1) *Overview*: A Multi-Layer Perceptron is a class of feedforward artificial neural networks consisting of at least three layers of nodes: an input layer, one or more hidden layers, and an output layer. MLPs utilize activation functions to introduce non-linearity into the model.

2) *Mathematical Formulation*:

3) *Forward Pass*: For a single hidden layer MLP, the computations can be described as follows:

$$\mathbf{z} = \mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)} \quad (11)$$

$$\mathbf{a} = \sigma(\mathbf{z}) \quad (12)$$

$$\mathbf{y} = \mathbf{W}^{(2)}\mathbf{a} + \mathbf{b}^{(2)} \quad (13)$$

Where:

- $\mathbf{x} \in R^n$ is the input vector.
- $\mathbf{W}^{(1)} \in R^{m \times n}$ and $\mathbf{b}^{(1)} \in R^m$ are the weights and biases for the hidden layer.
- σ is the activation function (e.g., ReLU, sigmoid, tanh).
- $\mathbf{a} \in R^m$ is the activation of the hidden layer.
- $\mathbf{W}^{(2)} \in R^{p \times m}$ and $\mathbf{b}^{(2)} \in R^p$ are the weights and biases for the output layer.
- $\mathbf{y} \in R^p$ is the output vector.

4) *Activation Function*: A commonly used activation function is the Rectified Linear Unit (ReLU):

$$\sigma(z) = \max(0, z) \quad (14)$$

Another popular choice is the sigmoid function:

$$\sigma(z) = \frac{1}{1 + e^{-z}} \quad (15)$$

5) *Training via Backpropagation*: Training MLPs involves adjusting the weights $\mathbf{W}^{(1)}$, $\mathbf{W}^{(2)}$ and biases $\mathbf{b}^{(1)}$, $\mathbf{b}^{(2)}$ to minimize a loss function (e.g., Mean Squared Error, Cross-Entropy Loss) using gradient-based optimization algorithms like Stochastic Gradient Descent (SGD).

$$\mathcal{L} = \frac{1}{N} \sum_{i=1}^N \mathcal{L}(y_i, \hat{y}_i) \quad (16)$$

Where:

- N is the number of training samples.
- $\mathcal{L}(y_i, \hat{y}_i)$ is the loss for the i -th sample.

6) *Backpropagation Algorithm*: The weights and biases are updated based on the gradient of the loss function with respect to each parameter:

$$\mathbf{W}^{(l)} := \mathbf{W}^{(l)} - \eta \frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(l)}} \quad (17)$$

$$\mathbf{b}^{(l)} := \mathbf{b}^{(l)} - \eta \frac{\partial \mathcal{L}}{\partial \mathbf{b}^{(l)}} \quad (18)$$

Where:

- l denotes the layer index.
- η is the learning rate.

E. Data set

We use the [14] data set to imitate their process as precise as possible. Where the architecture encoding and subsequently the input for the predictors are arrays of integer representations concerning the variables of the Convolutional Neural Networks (CNNs) with Peak Signal to Noise Ratio (PSNR) as output or ground truth if you will. The cardinality of one network encoding is 28 and the data set size is 1159. Where all networks are partially trained. More information on the partial training can be found in [14].

F. Gridsearch for hyperparameter optimization

As the 4 predictors all have hyperparameters that need be optimized, we performed a gridsearch, training with all combinations of the hyperparameters for all predictors except for the Radial Basis Function Network. The only hyperparameters to be optimized are the kernel type and tail function. However, due to the intrinsics of our data set the only possible combination is linear and constant as kernel type and tail function, respectively. Table I shows the gridsearch set-up and the results can be found at IV-F.

G. Preprocessing

We performed preprocessing on our data set to improve understandability of the inputs for our predictors. Showing significant improvements in all performance metrics of our predictors. We sorted the integer representation in numerical, ordinal and categorical data and used scikit-learn librarys standard scaler for the numerical and ordinal variables and a One-Hot encoder for the categorical variables. Then standard scaler normalizes the data around a mean of 0 with a standard deviation of 1. This prevents large variables to have too much influence

TABLE I: Predictors and Their Grid Search Parameters

Predictor	Hyperparameter	Grid Search Values
XGBoost	n_estimators	50, 100, 150, 200
	max_depth	3, 5, 7, 9
	learning_rate	0.05, 0.1, 0.15, 0.2
SVR	C	0.1, 1, 10
	gamma	scale, auto
	kernel	linear, rbf, poly
RBF	kernel	cubic, tps, linear
	tail	linear, constant
MLP	n_layers	1, 2, 3, 4
	n_hidden	50, 100, 300, 500
	drop	0.0, 0.1, 0.2, 0.4, 0.5
	lr	0.005, 0.001, 0.0008, 0.0005

on the predictability. The One-Hot-encoder transforms categorical variables in binary vectors, which increases readability of the data for the predictors.

H. Comparative analysis

To compare the four predictors in a precise and statistically sound manner we used several metrics. Firstly we used the Coefficient of Determination (R^2) to analyse the correlation between our predicted values and the ground truth:

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$

Where:

$$\bar{y} = \frac{1}{n} \sum_{i=1}^n y_i$$

Secondly, we used the Mean Squared Error (MSE) to quantify the deviation of our predictions compared to perfect predictions:

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

And Finally to assure large deficits are not overlooked in time complexity we measured the runtime of all predictors. Even though these majorly disproportionate to actual training and validating of the networks and should not pose problems.

IV. RESULTS

The results of our experiments, detailed in Tables 1 to 6, highlight notable differences in the performance of the predictors under various data splits and preprocessing conditions. The primary differences stem from the choice of train-test splits, which has a significant impact on the generalization of the surrogate models.

A. Train-Test Split Analysis

The results show that the predictors perform better with the **0.8-0.2 split** compared to the **0.964-0.036 split**, especially in terms of the R^2 scores. For instance, under the **0.8-0.2 split without pretraining** (Table 1), the best-performing model, RBF, achieves an R^2 of 0.3238, whereas under the **0.964-0.036 split without pretraining** (Table 4), the R^2 scores are generally lower, with the highest being 0.3373 for SVR. This highlights the importance of having a sufficiently large and representative test set for robust evaluation. The negative R^2 for MLP without pretraining with full data set combined with the positive R^2 without pretraining but a pretraining sized data set, is unusual and thus needs further investigation.

The **0.964-0.036 split**, while leading to seemingly low MSE values, suffers from an important issue: the test set is too small to reliably evaluate model performance. This is further evidenced by the visualizations in Figure 1 and Figure 2, which show that only a small number of data points are available for prediction under the smaller test split. As a result, the MSE appears to be low because the variability in the test set is small due to the low size of the test set, this artificially inflates the apparent performance of the models, which may not generalize well to new data.

B. Comparison to [14] Results

The findings also raise questions about the results reported by [14]. It is unclear how their reported performance metrics were achieved, particularly given their use of an unconventional train-test split. Their split (presumably similar to the **0.964-0.036 split**) heavily skews the data allocation toward training, leaving an inadequate amount of test data

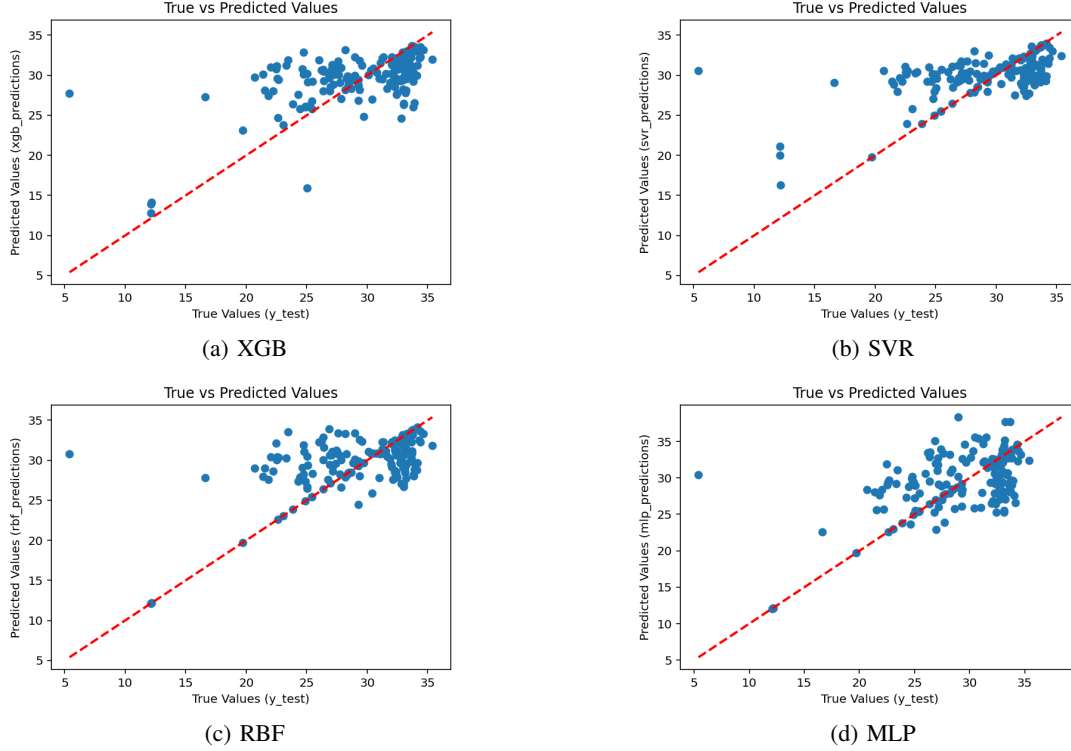


Fig. 1: True vs Predicted Values for the four predictors

to assess model generalization properly. This choice could result in misleading conclusions about the predictors’ effectiveness, as the performance metrics are derived from a non-representative subset of the data.

In our experiments, the **0.8-0.2 split** was chosen as it provides a better balance between training and testing, leading to more reliable evaluation of the predictors. The higher test R^2 scores and better alignment with expected behavior under this split underscore its suitability.

C. Insights from the Figures

The figures (Figure 1 and Figure 2) provide additional context for understanding the observed metrics. The scatter plots of true versus predicted values show that the predictors struggle to capture the full range of variability in the data. For the **0.964-0.036 split**, the limited test points exacerbate

this issue, as seen in the XGBoost visualization in Figure 2. The clustered nature of the test points minimizes the observed prediction error, which explains the deceptively low MSE values despite the lower R^2 .

D. Pretraining and Model Performance

When we compare the results of Tables 2,3 and 4 we find that pretraining generally has little impact to the XGBoost, SVR, and the RBF predictors, especially when you look at the tables 3 and 4, where the same amount of data but no pretraining has identical values for the XGBoost, SVR and the RBF, indicating that the size of the dataset is the determinant of the quality of the predictors. Only the MLP has an increase in performance from the pretraining, where the R^2 increases from -0.0566 to 0.2277 , but this increase also persists with the same amount of data but without pretraining. This

is likely due to the inability of the predictors to find correlation in FLOPs and PSNR, even though the FLOPs are normalized to PSNR range. Why the reduced data size improves the performance of especially the MLP may indicate that the model is better suited to smaller datasets, potentially due to reduced noise or simpler patterns in the data. However, further investigation is needed.

E. Runtime Analysis

The runtime differences across the predictors highlight the computational trade-offs. While RBF consistently achieves competitive performance in R^2 with the lowest runtime (e.g., 0.3444s in Table 1), MLP and XGBoost require significantly more time for training and prediction. This makes RBF an attractive choice for scenarios where computational efficiency is a priority.

Model	R^2	MSE	Runtime (s)
XGBoost	0.2536	10.5629	28.5415
SVR	0.2724	10.2975	16.2276
RBF	0.3238	9.57	0.3444
MLP	-0.0566	14.9534	39.0578

TABLE II: 0.8-0.2 split without pretraining

Model	R^2	MSE	Runtime (s)
XGBoost	0.311	16.6981	27.6229
SVR	0.3148	16.607	9.1616
RBF	0.2977	17.0193	0.3128
MLP	0.2277	18.7164	48.3301

TABLE III: 0.8-0.2 split with pretraining

Model	R^2	MSE	Runtime (s)
XGBoost	0.311	16.6981	25.4216
SVR	0.3148	16.607	9.1633
RBF	0.2977	17.0193	0.2735
MLP	0.2457	18.2801	30.4385

TABLE IV: 0.8-0.2 split without pretraining, same data size as with pretraining

F. Best found Parameters for Models via Grid-search

1) *XGBoost*: The best parameters for the XGBoost model are presented in the table below:

learning_rate	max_depth	n_estimators
0.05	5	150

Model	R^2	MSE	Runtime (s)
XGBoost	0.2195	6.6138	30.5715
SVR	0.3373	5.6157	20.7771
RBF	0.3094	5.8516	0.4145
MLP	-0.4225	12.0533	47.2315

TABLE V: 0.964-0.036 split without pretraining

Model	R^2	MSE	Runtime (s)
XGBoost	0.0798	30.5277	29.0443
SVR	-0.0467	34.7257	11.0134
RBF	0.0205	32.4969	0.3247
MLP	-0.1609	38.5159	53.5003

TABLE VI: 0.964-0.036 split with pretraining

Model	R^2	MSE	Runtime (s)
XGBoost	0.0798	30.5277	28.8208
SVR	-0.0467	34.7257	10.9885
RBF	0.0205	32.4969	0.3153
MLP	-0.0745	35.6468	35.9241

TABLE VII: 0.964-0.036 split without pretraining, same data size as with pretraining

2) *SVR*: The best parameters for the SVR model are presented in the table below:

C	gamma	kernel
10	scale	poly

3) *MLP*: The best parameters for the MLP model are presented in the table below:

n_layers	n_hidden	drop	lr
3	500	0.2	0.005

V. CONCLUSIONS

In conclusion, our results demonstrate that the choice of train-test split plays a critical role in determining the performance and reliability of surrogate models in NAS. The **0.8-0.2 split** provides a more balanced and robust evaluation, highlighting the limitations of smaller test splits like **0.964-0.036**. Moreover, while our findings validate some of the trends reported by [14], the discrepancies due to different train-test splits emphasize the need for careful experimental design to ensure reproducible and generalizable results.

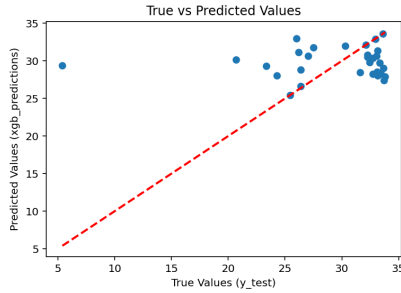


Fig. 2: XGB with 0.036 test size

REFERENCES

- [1] Hadjer Benmeziiane. Efficient hardware-aware neural architecture search for edge computing. <https://tel.archives-ouvertes.fr/tel-04224035>, 2023. Machine Learning [cs.LG], ffnnt : 2023UPHF0022ff.
- [2] Ronghong Cai and Jianping Luo. Multi-task learning for multi-objective evolutionary neural architecture search. In *Proceedings of the 1st Conference on Neural Architecture Search*, Shenzhen, China, 2024. Email: ljp@szu.edu.cn.
- [3] Y.-C. Chen, C. Gao, E. Robb, and J.-B. Huang. Nas-dip: Learning deep image prior with neural architecture search. In A. Vedaldi, H. Bischof, T. Brox, and J.-M. Frahm, editors, *Computer Vision – ECCV 2020*, pages 442–459, Cham, 2020. Springer International Publishing.
- [4] X. Chu, B. Zhang, H. Ma, R. Xu, and Q. Li. Fast, accurate and lightweight super-resolution with neural architecture search. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops*, 2020.
- [5] Weisheng Dong, Peiyao Wang, Wotao Yin, Guangming Shi, Fangfang Wu, and Xiaotong Lu. Denoising prior driven deep neural network for image restoration. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 41(10):2305–2318, 2019.
- [6] W. E. R. Hu, and S. Peng. Deep learning in finance. *Digit Finance*, 5:1–2, March 2023.
- [7] Amirhosein Ghasemabadi, Benyamin Mehmandar, and Ahmad Kalhor. Autonomous system identification and control using deep neural network, 2021.
- [8] Bryson Greenwood and Tyler McDonnell. Surrogate-assisted neuroevolution. *SparkCognition*, 2024. Email: tmcdonnell@sparkcognition.com.
- [9] Dongkyu Kim, Han-Ul Jang, Seung-Min Mun, Sunghee Choi, and Heung-Kyu Lee. Median filtered image restoration and anti-forensics using adversarial networks. *IEEE Signal Processing Letters*, 25(2):278–282, 2018.
- [10] Rafael Molina, José Luis Nuñez, Francisco J. Cortijo, and Javier Mateos. Image restoration in astronomy: a bayesian perspective. *IEEE Signal Process. Mag.*, 18:11–29, 2001.
- [11] Chong Mou, Qian Wang, and Jian Zhang. Deep generalized unfolding networks for image restoration, 2022.
- [12] E. Real, A. Aggarwal, Y. Huang, and Q. V. Le. Regularized evolution for image classifier architecture search. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 33, pages 4780–4789, 2019.
- [13] E. Real, S. Moore, A. Selle, S. Saxena, Y. L. Suematsu, J. Tan, Q. V. Le, and A. Kurakin. Large-scale evolution of image classifiers. In *International Conference on Machine Learning*, pages 2902–2911. PMLR, 2017.
- [14] Sergio Sarmiento-Rosales, Jesus Leopoldo Llano García, Jesus Guillermo Falcón-Cardona, Raul Monroy, Manuel Ivan Casillas del Llano, and Víctor Adrian Sosa Hernández. Surrogate modeling for efficient evolutionary multi-objective neural architecture search in super resolution image restoration. In *Proceedings of the Evolutionary Computation Conference*, 2024. Emails: a01801059, a01748867, jfalcon, raulm, manuel.casillas, vsosa@tec.mx.
- [15] Nida Shahid, Tim Rappon, and Whitney Berta. Applications of artificial neural networks in health care organizational decision-making: A scoping review. *PLOS ONE*, 14(2):1–22, 02 2019.
- [16] Jae Woong Soh and Nam Ik Cho. Variational deep image restoration. *IEEE Transactions on Image Processing*, 31:4363–4376, 2022.
- [17] Emma Strubell, Ananya Ganesh, and Andrew McCallum. Energy and policy considerations for deep learning in nlp. *arXiv preprint arXiv:1906.02243*, 2019. Presented at the 57th Annual Meeting of the Association for Computational Linguistics (ACL), Florence, Italy, July 2019.
- [18] F. Tatsugami, T. Higaki, Y. Nakamura, Z. Yu, J. Zhou, Y. Lu, C. Fujioka, T. Kitagawa, Y. Kihara, M. Iida, and K. Awai. Deep learning-based image restoration algorithm for coronary ct angiography. *European Radiology*, 29(10):5322–5329, 2019.
- [19] W. W. Z. Wilman and P. C. Yuen. Very low resolution face recognition problem, 2010.
- [20] H. Zhang, Y. Dai, H. Li, and P. Koniusz. Deep stacked hierarchical multipatch network for image deblurring, 2019.
- [21] B. Zoph, V. Vasudevan, J. Shlens, and Q. V. Le. Learning transferable architectures for scalable image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 8697–8710, 2018.